# TP4 : le format Targa (TGA) et la compression RLE

R3.L01 — Représentations et codages des images

Fiche d'explication du TP, écrite à partir du cours (CM diapos 15 à 21, 40, 57 et 59), du sujet du TP4, de `TGA.pdf`, `tgaffs.pdf` et de la fiche « Manipulations des nombres entiers binaires ».

Code complet et testé : `PROJET_FINAL/Image.cpp` (writeTGA, readTGA), `tp4.cpp`

# Ce qu'il faut faire

## Les deux fonctions demandées

```
void ColorImage::writeTGA(std::ostream& f, bool rle = true)
const;
static ColorImage* ColorImage::readTGA(std::istream& f);
```

1. Écriture puis lecture du **sous-format 2** : RGB 24 bits non compressé.
2. Lecture du **sous-format 1** : image avec palette 24 bits, non compressée. L'image est convertie en RGB (notre `ColorImage` ne garde pas de palette).
3. Écriture du **sous-format 10** : RGB compressé en RLE, quand `rle` vaut vrai (par défaut).

On ne gère que les couleurs 24 bits. Tout autre cas (16 ou 32 bits, sous-format inconnu...) doit lever une **exception dérivée de `std::exception`** : « il n'est pas acceptable que votre programme ne fasse rien » (sujet).

| type | contenu (TGA.pdf) |
|---|---|
| 0 | pas d'image |
| **1** | **palette, non compressé** |
| **2** | **RGB, non compressé** |
| 3 | noir et blanc, non compressé |
| 9 | palette, RLE |
| **10** | **RGB, RLE** |
| 11 | noir et blanc, compressé |

```
cd PROJET_FINAL
make tp4 && ./tp4
```

# L'en-tête : 18 octets binaires (et plus de texte !)

| octet | taille | champ | chat.tga |
|---|---|---|---|
| 0 | 1 | longueur du champ d'identification | `00` |
| 1 | 1 | type de palette (0 : aucune, 1 : présente) | `00` |
| 2 | 1 | **sous-format** (1, 2, 10...) | **`02`** |
| 3 | 2 | palette : indice de la 1re entrée | `00 00` |
| 5 | 2 | palette : nombre d'entrées | `00 00` |
| 7 | 1 | palette : bits par entrée (24) | `00` |
| 8 | 2 | X origine | `00 00` |
| 10 | 2 | Y origine | `00 00` |
| 12 | 2 | **largeur** | **`40 01`** = 0x0140 = 320 |
| 14 | 2 | **hauteur** | **`f0 00`** = 0x00f0 = 240 |
| 16 | 1 | **bits par pixel** | 18 = 24 |
| 17 | 1 | **descripteur** (bit 5 : origine en haut) | `20` |

Ensuite : le champ d'identification (sa longueur est l'octet 0, souvent 0), la palette s'il y en a une, puis les pixels.

Contrairement à PGM/PPM, **rien n'est en texte** : 320 est stocké sur 2 octets binaires. Pas de magic number non plus : il faut vérifier chaque champ.

TGA brut 320 × 240 : 18 + 320 × 240 × 3 = **230 418** octets.

# Les entiers de 16 bits : little endian (« lo-hi ») CM diapos 15 à 19

largeur = 320 = 0x0140




On pourrait écrire `os.write((const char*)&width, 2)` : ça marche sur un PC Intel (little endian)... et donne un fichier faux sur une machine big endian. Avec des **décalages et des masques**, le code est correct sur toutes les machines (TD2, exercice 2, question 3).

```
// Écrit un entier 16 bits en little endian,
// quel que soit le processeur.
static void write_uint16_le(std::ostream& os, uint16_t
v)
{
  os.put(char(v & 0xFF)); // octet de poids faible
d'abord
  os.put(char(v >> 8));   // puis octet de poids fort
}

// Relit un entier 16 bits little endian.
static uint16_t read_uint16_le(const uint8_t* p)
{ return uint16_t(p[0] | (p[1] << 8)); }
```

## Les opérateurs bit à bit (fiche du prof)

`0x0140 & 0xFF` = 0x40 : le masque garde les 8 bits de droite.
`0x0140 >> 8` = 0x01 : décaler de 8 rangs à droite = diviser par $2^8$.
`0x40 | (0x01 << 8)` = 0x40 | 0x100 = 0x140 : on recolle les deux octets.

# Le descripteur (octet 17) : dans quel sens sont les lignes ?

| 7 | 6 | 5 | 4 | 3 | 2 | 1 | 0 | |
|---|---|---|---|---|---|---|---|---|
| 0 | 0 | 1 | 0 | 0 | 0 | 0 | 0 | = 0x2 |

7-6 : entrelacement (00 = non)

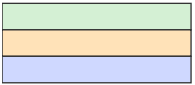
5 : 1 = 1re ligne écrite = ligne du HAUT

4 : 1 = pixels de droite à gauche

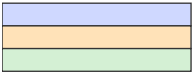
3-0 : bits d'attribut (alpha), 0 en 24 bits

```
const bool topToBottom = (descriptor & 0x20) != 0; // bit 5
if (descriptor & 0x10)    // bit 4
  throw std::runtime_error("readTGA : ordre de droite à gauche non géré");
if (descriptor & 0xC0)    // bits 7 et 6
  throw std::runtime_error("readTGA : images entrelacées non gérées");
```

bit 5 = 0 (palette_bl.tga, chanel.tga)




bit 5 = 1 (chat.tga, nos fichiers)




Par défaut, TGA range les lignes **de bas en haut** (comme un repère de maths). Le *n*-ième pixel lu va donc en :

```
x   = n % width;
row = n / width;              // rang de la ligne dans le fichier
y   = topToBottom ? row : height - 1 - row;
```

Nous écrivons avec le bit 5 à 1 (de haut en bas, comme notre tableau). `palette_bl.tga` et `palette_tl.tga` donnent bien la même image à l'endroit.

# Les pixels : BGR, et la palette du sous-format 1 CM diapos 40-41

## Attention à l'ordre des composantes

Un pixel 24 bits TGA est stocké **bleu, vert, rouge** (« 1 byte each of blue, green and red »), l'inverse du PPM. On ne peut donc pas écrire le tableau de `Color` d'un seul bloc.

```
static void write_bgr(std::ostream& os, const
Color& c)
{
  os.put(char(c.b));
  os.put(char(c.g));
  os.put(char(c.r));
}
// lecture : Color(bgr[2], bgr[1], bgr[0])
```

L'octet 16 vaut 24 (pixels RGB) ou 8 (indices de palette). Les cas 16 et 32 bits lèvent une exception.

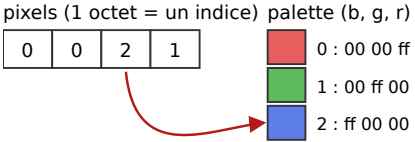


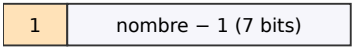

image RGB obtenue après lecture

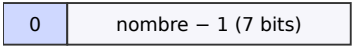
- La palette suit l'en-tête : *nombre d'entrées* × 3 octets b, g, r.
- Un indice `v` désigne l'entrée `v – origine` (octets 3-4, souvent 0).
- Indice hors de la palette = fichier invalide → exception.
- `chat010couleurs.tga` : palette de 10 couleurs, 1 octet par pixel.

# La compression RLE de TGA (sous-format 10) CM diapo 59

**Principe (CM)** : une suite de pixels identiques est remplacée par (nombre, valeur) ; une suite de pixels différents par (nombre, valeurs).

Dans TGA, chaque **paquet** commence par un octet d'en-tête :




bit 7 : type du paquet ; 7 bits → au plus 127 + 1 = 128 pixels

```
int packet = is.get();
size_t count = (packet & 0x7F) + 1;   // 7 bits de
droite, + 1
if (packet & 0x80)  { /* 1 couleur, répétée count fois
*/ }
else                { /* count couleurs différentes
*/ }
```

## Exemple : une ligne de 7 pixels

R R R R V B B (R = rouge, V = vert, B = bleu)

| paquet | octets écrits |
|---|---|
| 4 × R répété | `83 \| 00 00 ff` |
| 1 pixel brut | `00 \| 00 ff 00` |
| 2 × B répété | `81 \| ff 00 00` |

12 octets au lieu de 7 × 3 = 21. `0x83` = 1000 0011 : bit 7 à 1, 3 + 1 = 4 pixels.

**Cas défavorable** : une photo où les pixels voisins sont presque tous différents. Les paquets bruts ajoutent 1 octet tous les 128 pixels au plus : le fichier peut même grossir un peu. RLE est efficace « lorsque le nombre de couleurs est limité » (CM).

# writeTGA : le code

```
void ColorImage::writeTGA(std::ostream& os, bool rle) const
{
  os.put(0);                    // 0 : pas d'identification
  os.put(0);                    // 1 : pas de palette
  os.put(char(rle ? 10 : 2));   // 2 : sous-format
  write_uint16_le(os, 0);       // 3-4 : palette (origine)
  write_uint16_le(os, 0);       // 5-6 : palette (longueur)
  os.put(0);                    // 7 : palette (bits)
  write_uint16_le(os, 0);       // 8-9 : X origine
  write_uint16_le(os, 0);       // 10-11 : Y origine
  write_uint16_le(os, width);   // 12-13 : largeur
  write_uint16_le(os, height); // 14-15 : hauteur
  os.put(24);                   // 16 : bits par pixel
  os.put(0x20);                 // 17 : ligne du haut
d'abord
  for (uint16_t y = 0; y < height; y++) {
    if (!rle) {
      for (uint16_t x = 0; x < width; x++)
        write_bgr(os, pixel(x, y));
      continue;
    }
    ...   // paquets RLE de la ligne y (à droite)
  }
  if (!os)
    throw std::runtime_error("writeTGA : erreur
d'écriture");
}
```

```
// Les paquets restent dans la ligne y.
uint16_t x = 0;
while (x < width) {
  // pixels identiques à pixel(x,y) ? (au plus 128)
  uint16_t run = 1;
  while (x + run < width && run < 128
         && pixel(x + run, y) == pixel(x, y))
    run++;
  if (run >= 2) {    // paquet répété
    os.put(char(0x80 | (run - 1)));  // bit 7 à 1
    write_bgr(os, pixel(x, y));
    x += run;
  } else {           // paquet brut : on avance tant
    uint16_t n = 1;  // qu'aucune répétition ne commence
    while (x + n < width && n < 128 &&
           !(x + n + 1 < width &&
             pixel(x + n, y) == pixel(x + n + 1, y)))
      n++;
    os.put(char(n - 1));             // bit 7 à 0
    for (uint16_t k = 0; k < n; k++)
      write_bgr(os, pixel(x + k, y));
    x += n;
  }
}
```

# readTGA (1/2) : lire et vérifier l'en-tête

```
uint8_t header[18];
if (!is.read((char*)header, 18))
  throw std::runtime_error("readTGA : en-tête incomplet");
const uint8_t  idLength   = header[0];
const uint8_t  cmapType   = header[1];
const uint8_t  imageType  = header[2];
const uint16_t cmapOrigin = read_uint16_le(header + 3);
const uint16_t cmapLength = read_uint16_le(header + 5);
const uint8_t  cmapBits   = header[7];
const uint16_t w          = read_uint16_le(header + 12);
const uint16_t h          = read_uint16_le(header + 14);
const uint8_t  pixelBits  = header[16];
const uint8_t  descriptor = header[17];
// Tous les cas que l'on ne sait pas traiter lèvent une exception (sujet du TP4).
if (imageType != 1 && imageType != 2 && imageType != 10)
  throw std::runtime_error("readTGA : sous-format non géré (seuls 1, 2 et 10 le sont)");
if (imageType == 1 && (cmapType != 1 || pixelBits != 8))
  throw std::runtime_error("readTGA : sous-format 1 attendu avec palette et indices 8 bits");
if (imageType != 1 && pixelBits != 24)
  throw std::runtime_error("readTGA : seules les couleurs 24 bits sont gérées");
if (cmapType > 1 || (cmapType == 1 && cmapBits != 24))
  throw std::runtime_error("readTGA : seules les palettes 24 bits sont gérées");
// ... + bits 4, 6, 7 du descripteur (page « descripteur ») et dimensions nulles
// La suite est dans un try : palette et image sont libérées si la lecture échoue.
```

# readTGA (2/2) : palette puis pixels

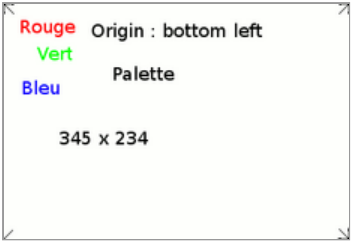
```
Color* palette = nullptr; ColorImage* res = nullptr;
try { skip_bytes(is, idLength);          // champ d'identification
  if (cmapType == 1) {                   // cmapLength entrées b, g, r
    palette = new Color[cmapLength];
    for (uint16_t i = 0; i < cmapLength; i++) palette[i] = read_tga_bgr(is);
  }
  res = new ColorImage(w, h); size_t n = 0; // n : rang du pixel lu
  if (imageType == 10) {                 // paquets RLE
    while (n < total) {
      const uint8_t packet = read_byte(is);
      size_t count = (packet & 0x7F) + 1;
      if (n + count > total) throw std::runtime_error("RLE : dépasse");
      if (packet & 0x80) {               // 1 couleur répétée count fois
        Color c = read_tga_bgr(is);
        for (size_t k = 0; k < count; k++)
          store_tga_pixel(*res, n++, topToBottom, c);
      } else {                           // count couleurs différentes
        for (size_t k = 0; k < count; k++)
          store_tga_pixel(*res, n++, topToBottom, read_tga_bgr(is));
      }
    }
  } else if (imageType == 1) {           // palette, non compressé
    for (n = 0; n < total; n++)
      store_tga_pixel(*res, n, topToBottom,
        read_tga_index(is, palette, cmapLength, cmapOrigin));
  } else {                               // type 2, non compressé
    for (n = 0; n < total; n++)
      store_tga_pixel(*res, n, topToBottom, read_tga_bgr(is));
  }
} catch (...) { delete[] palette; delete res; throw; } // pas de fuite
delete[] palette; return res;
```

- `read_tga_bgr` lit 3 octets b, g, r ; `read_tga_index` lit 1 indice de palette (type 1). Ils lèvent une exception si le fichier est tronqué ou l'indice invalide.
- `store_tga_pixel` range le *n*-ième pixel lu en `(n % w, ligne)`, en retournant l'image si l'origine est en bas.
- `total` = w × h.
- Le `catch (...)` libère la palette et l'image si quoi que ce soit échoue, puis relance l'exception.

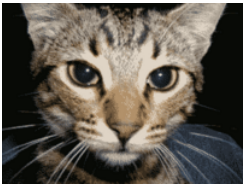
# Résultats et taux de compression

| image | brut (type 2) | RLE (type 10) | taux |
|---|---|---|---|
| chat.tga (photo) | 230 418 o | 207 106 o | 1,1 : 1 |
| chat + bandeau jaune de 100 lignes | 230 418 o | 122 672 o | 1,9 : 1 |
| chat010couleurs.tga (10 couleurs) | 230 418 o | 104 396 o | 2,2 : 1 |
| palette_bl.tga (texte sur fond uni) | 242 208 o | 11 985 o | 20 : 1 |

- Sur une photo, RLE ne gagne presque rien : les voisins sont rarement identiques.
- Sur une image à peu de couleurs, avec de grandes zones unies, c'est très efficace.
- Vérifié : chaque fichier écrit est relu à l'identique par notre `readTGA` **et** par ImageMagick (0 pixel différent), et `chanel.tga` (fourni, en RLE et de bas en haut) se lit correctement.




palette_bl.tga relu puis réécrit : à l'endroit.



chat010couleurs.tga (palette de 10 couleurs)

# Ce qui a été corrigé par rapport à la version de Gemini

| Point | Version Gemini | Correction |
|---|---|---|
| Contenu du dossier TP4 | **simpleScale, bilinearScale, writeJPEG, readJPEG ont disparu**, et la lecture PPM a perdu ses contrôles d'erreur. | Un seul projet qui garde tout le travail des TP1 à TP5. |
| Lecture des pixels | `f.get(...)` sans vérification : un fichier tronqué donne une image remplie de valeurs au hasard, sans erreur. | Chaque lecture est vérifiée → `runtime_error("fichier tronqué")`. |
| Fuite mémoire | Exception (indice de palette invalide) après le `new` : l'image n'est jamais libérée. | `try { ... } catch (...) { delete res; throw; }` |
| Paquet RLE trop long | Non vérifié (écriture hors de l'image... arrêtée par `pixel()`, mais avec fuite). | Vérifié avant de décoder. |
| Descripteur | Bits 4, 6 et 7 ignorés. | Exception si droite à gauche ou entrelacé. |

# Questions possibles au contrôle

1. **Lire la largeur dans l'en-tête `... 40 01 f0 00 ...`** : 0x0140 = 320 (little endian) ; hauteur 0x00f0 = 240.
2. **Taille d'un TGA type 2 de 100 × 50 ?** 18 + 100 × 50 × 3 = 15 018 octets.
3. **Que vaut l'octet d'en-tête d'un paquet de 10 pixels identiques ?** 0x80 | 9 = 0x89.
4. **Et de 128 pixels bruts ?** 0x7F.
5. **Décoder `82 ff 00 00 | 01 00 00 00 ff ff ff`** : 3 pixels bleus, puis 2 pixels bruts : noir, blanc.
6. **Tester si l'origine est en haut** : `(descriptor & 0x20) != 0`.
7. **Pourquoi ne pas écrire `array` en bloc ?** TGA attend b, g, r et notre `Color` est r, g, b.
8. **Taille d'une image 320 × 240 avec palette de 256 couleurs ?** 18 + 256 × 3 + 320 × 240 = 77 586 octets, ≈ 3 fois moins qu'en 24 bits.